Adaptive Advance-Time Control for nFAPI Split under Unpredictable Latency

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1†
*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan
†Affiliation 2
Email: crg@mail.ntust.edu.tw

Abstract—
Index Terms—nFAPI, O-RAN, Functional Split, Adaptive Control.

I. Introduction

A. Background and Motivation

The evolution of fifth-generation (5G) mobile networks toward open and disaggregated architectures has fundamentally transformed the design of Radio Access Networks (RANs). Initiatives such as Open RAN (O-RAN) and 3GPP-defined functional splits [1] enable network functions to be distributed across heterogeneous platforms, thereby improving deployment flexibility, vendor interoperability, and cost efficiency. However, this architectural shift introduces a critical and often underaddressed challenge: maintaining precise timing coordination across physically separated network entities under non-deterministic transport conditions.

In traditional monolithic base stations, inter-layer interactions are tightly coupled within a single hardware platform, where timing relationships are inherently deterministic. In contrast, functional split architectures replace internal data buses with packet-based transport networks, where latency, jitter, and processing delays become non-deterministic [2]. As a result, synchronization is no longer a purely physical-layer problem, but evolves into a cross-layer system challenge involving transport protocols, scheduling logic, and timing window management.

Although multiple functional split options have been standardized, including 3GPP Option 2, Small Cell Forum (SCF) Option 6 (nFAPI), and O-RAN Option 7.2x, they fundamentally differ in their synchronization models and timing sensitivities. Option 2, which separates the Packet Data Convergence Protocol (PDCP) and Radio Link Control (RLC) layers, operates over the F1 interface and primarily relies on message-based coordination and status reporting. Its timing requirements are relatively relaxed, typically at the millisecond level, and timing misalignment is mitigated through buffering and retransmission mechanisms.

In contrast, Option 6 introduces a tighter coupling between the Medium Access Control (MAC) and Physical (PHY) layers through the nFAPI interface [3]. In this architecture, per-slot scheduling decisions must be delivered within strict deadlines, requiring slot-level synchronization between distributed entities. Synchronization is achieved through timestamp-based message exchanges (e.g., P7 Node Sync), enabling delay estimation and timing alignment. However, unlike lower-layer splits, Option 6 operates over general-purpose transport networks (e.g., Ethernet), where latency variations caused by operating system scheduling and network congestion can significantly disrupt timing determinism.

Option 7.2x further pushes the limits of synchronization by splitting the PHY layer into high-PHY and low-PHY components and transporting time-sensitive in-phase and quadrature (IQ) samples over the fronthaul [4]. This architecture requires nanosecond-level time and phase synchronization, typically achieved through hardware-assisted mechanisms such as IEEE 1588 Precision Time Protocol (PTP) and Synchronous Ethernet (SyncE) [5]. To preserve real-time processing constraints, strict timing windows (e.g., T_{2a} and T_{a4}) are enforced, and packets arriving outside these windows are typically discarded.

These distinct synchronization paradigms also lead to fundamentally different timing window behaviors. In Option 7.2x, timing windows are rigid and enforced at the physical layer, where out-of-window packets are dropped to ensure real-time signal processing. In Option 2, timing misalignment is largely absorbed by higher-layer retransmission and buffering mechanisms, with minimal strict enforcement. Option 6, however, occupies a unique intermediate position: it requires strict slot-level timing alignment but operates over non-deterministic transport networks without hardware-enforced synchronization. Consequently, timing violations often manifest as scheduling failures (e.g., missing subframes), and existing implementations typically rely on static timing configurations that are insufficient under dynamic network conditions.

This gap highlights a fundamental limitation in current disaggregated RAN designs: while synchronization mechanisms exist at both extremes (message-level and hardware-level), there is a lack of adaptive timing control mechanisms for intermediate-layer splits operating under non-ideal transport environments.

B. Related Works

The evolution of Radio Access Network (RAN) architectures from Distributed RAN (D-RAN) to Open RAN (O-RAN) requires a thorough study of functional splits [2], [6], [7]. The MAC-PHY split (Option 6), standardized by the Small Cell Forum (SCF) as nFAPI [3], [8], is suitable for small cell deployments. It allows the MAC layer to be virtualized on commercial servers while the PHY layer remains on dedicated hardware.

The performance of the nFAPI split depends on the transport network. Practical deployments often use non-ideal fronthaul, such as Shared Ethernet or DOCSIS. Mehran and MacKenzie [9] demonstrated that uplink throughput drops to zero with a small increase in fronthaul delay. Furthermore, server processing delay is unpredictable. Khalaf et al. [10] indicated that server delay increases under high CPU utilization, causing the MAC scheduler to miss its timing window.

To address fronthaul latency, Huang and Zhang [11] proposed a distributed MAC scheduling scheme. While this scheme reduces the impact of delayed grants, it introduces a "split-brain" issue with multiple schedulers. Akgun et al. [12] utilized intelligent scheduling to adapt modulation based on interference, but did not solve the sub-millisecond timing of message delivery itself. Existing OpenAirInterface (OAI) reference models typically rely on static timing windows, leading to frequent "missing subframe" errors when network jitter exceeds configured limits. This paper bridges this gap by proposing a centralized Adaptive Advance-Time Control algorithm to dynamically manage scheduling timing.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

We identify that the root cause of performance degradation in such systems lies in the inability of the MAC scheduler to adapt its transmission timing to dynamic end-to-end latency variations. Existing open-source implementations, such as OpenAirInterface (OAI), typically employ static timing advance configurations, which fail to accommodate fluctuating network conditions, leading to frequent synchronization failures and throughput degradation.

To address this issue, we propose an adaptive timing control framework that dynamically adjusts scheduling lead time based on real-time latency estimation. The proposed approach introduces a dual-loop architecture, where a convergence optimization loop estimates end-to-end delay characteristics, and a timing actuator dynamically compensates for timing offsets to ensure that scheduling messages arrive within the valid timing window.

The main contributions of this paper are summarized as follows:

- Systematic Analysis of Timing Uncertainty in Split 6: We characterize the impact of non-deterministic latency, including network jitter and processing delay, on synchronization stability in disaggregated RAN systems.
- Adaptive Advance-Time Control Algorithm: We propose a dynamic timing adjustment mechanism that continuously aligns scheduling decisions with varying transport delays, effectively mitigating timing violations.
- Experimental Validation on OAI Platform: We implement the proposed framework in a real-world testbed and demonstrate that it eliminates missing subframe errors and significantly improves throughput stability under non-ideal fronthaul conditions.

The rest of the paper is organized as follows. Section II presents the system model and architecture. Section III describes the proposed adaptive timing control mechanism. Section IV evaluates the performance through experiments. Finally, Section VI concludes the paper.

II. System Model

Before diving into the system details, this study establishes a fundamental assumption: currently, there is a lack of Open Radio Access Network (O-RAN) Radio Units (O-RUs) in the market that natively support the Network Functional Application Platform Interface (nFAPI) via Split 6. As a result, the proposed system adopts a resilient two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to the remote O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- Core Network (CN): The central element responsible for routing, session management, and authenticating user data.
- O-RAN Central Unit (O-CU): A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- O-RAN Distributed Unit (O-DU): To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized

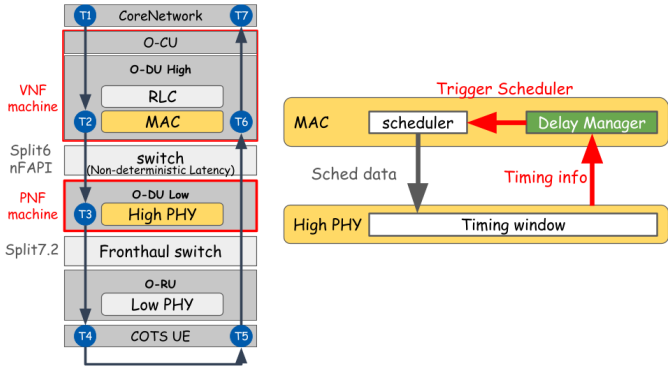

Fig. 1. System architecture and delay management mechanism.

machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.

- Switch: A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- O-RAN Radio Unit (O-RU): A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- Commercial Off-The-Shelf User Equipment (COTS UE): Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC and Delay Manager

Within the Media Access Control (MAC) layer of the O-DU High, the core components handling timing mechanisms include the Scheduler and the nFAPI Delay Manager:

- Scheduler: Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- nFAPI Delay Manager: This resides as the crucial control unit of this delay-aware architecture. Because unpredictable, unstable latency exists in the switch between the VNF and PNF, the Delay Manager continuously receives timing feedback from the lower physical layer. It uses this to dynamically calculate network jitter and proactively trigger the scheduler. This calculation guarantees that the scheduling instructions (Sched data) are generated ahead of time to account for network delays.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- Timing Window: A predefined, strict time duration. When the scheduling message (specifically the DL_TTI.request) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the subframe data.
- Timing Info: The PNF continuously monitors the exact arrival timestamp of these control messages and periodically sends this "Timing Info" back to the Delay Manager in the MAC layer. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following three Key Validation Metrics, where RTT denotes the Round-Trip Time:

- Ping RTT ($T_7 - T_1$): Measures the end-to-end round-trip latency at the application layer. This metric captures the complete duration from when a packet is sent from the CN to when its return acknowledgment is received by the sender.
- HARQ RTT ($T_6 - T_2$): Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. HARQ RTT accurately reflects the feedback efficiency and operational health of the link layer.
- VNF-PNF Downlink Latency ($T_3 - T_2$): Measures the one-way downlink latency accumulated from the virtualized machine to the physical machine. This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

To validate the proposed performance optimization within a realistic network environment, the following assumptions and system design constraints are established:

- Predominance of O-RAN Split 7.2 in Commercial Infrastructure: While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [13], current commercial-grade O-RUs primarily support Split 7.2x. This creates a barrier for direct nFAPI deployment in commercial ecosystems.
- Research Prototypes and FPGA-based nFAPI Implementations: Current literature on nFAPI-compliant RUs is largely restricted to experimental prototypes or FPGA-based development boards [14], [15]. Open-source stacks like OpenAirInterface have incorporated 5G NR nFAPI support [16], yet require specialized hardware adapters to interface with mainstream RUs.
- Hybrid System Architecture for E2E Validation: Given the lack of native Option 6 support in commercial O-RUs, this paper adopts a hybrid architecture

(Split Option 6 + Split Option 7.2). This facilitates an interworking layer that enables the nFAPI-driven MAC/High-PHY to interact with commercial O-RUs, allowing end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. Proposed Method

This section details the proposed dynamic timing adjustment algorithm. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose a Dual-Loop Timing Control Architecture.

A. Dual-Loop Architecture

The timing control of this system adopts a "policy-execution separation" dual-loop architecture, ensuring that the timing advancement on the VNF side can flexibly cope with network jitter without inducing unexpected oscillations:

1) Convergence Optimization Loop (Policy): Dynamically analyzes the statistical data reported by the PNF through the convergence optimization algorithm to determine the global target slot-ahead s_{ahead}.
2) Timing Actuator Loop (Execution): Executed by an independent VNF timing thread. This thread is the only unit in the system capable of altering the absolute sleep time and slot propagation, translating the upper-layer policy target and low-layer microsecond phase deviations into actual timing advancement.

B. VNF Timing Actuator

The core function of the VNF Timing Actuator is isolation and precise phase execution. Initially, we define the basic mathematical parameters used in the system. Let μ denote the subcarrier spacing parameter (Numerology), and the corresponding slot duration (in microseconds) is defined as T_{slot}:

$$T_{\text{slot}} = \frac{1000}{2^{\mu}} \tag{1}$$

Let s_{ahead} be the target slot-ahead value, and T_{adv} be the accumulated total phase advance in microseconds. For microsecond-level phase correction, let Δ_{slot} be the integer slot correction provided by the lower layer, and Δ_{pending} be the sub-slot phase deviation (in μs). The algorithm calculates the next absolute wake-up time T_{next}:

$$T_{\text{next}}^{(k)} = T_{\text{next}}^{(k-1)} + T_{\text{slot}} + \Delta_{\text{pending}} \tag{2}$$

Before receiving the initial timing anchor from the PNF, the thread blocks execution via a condition variable. Once active, the thread operates within a mutex-locked environment and triggers sleep according to the absolute monotonic clock calculated in (2), mitigating long-term error accumulation. The condensed control logic is presented in Algorithm 1.

Algorithm 1 VNF Timing Actuator Thread

```
procedure TimingActuator(s_ahead, μ)
    Wait for timing_info condition variable
    T_next ← Now()
    ID_DEC ← DEC(μ, sfn, slot)
    while running do
        Lock Mutex
        T_adv ← s_ahead · T_slot
        if Δ_slot ≠ 0 then                          ▷ Integer correction
            ID_DEC += Δ_slot
            if sync_locked then
                T_adv += Δ_slot
            end if
            Δ_slot ← 0
        end if
        Δ_current ← Δ_pending                       ▷ Sub-slot correction
        if sync_locked then
            T_adv −= Δ_current
        end if
        Δ_pending ← 0
        Unlock Mutex
        T_next += T_slot + Δ_current
        SleepUntil(T_next)
        HandlePeriodicSync
        target ← wrap(ID_DEC + s_ahead)
        while last_mac ≠ target do                  ▷ Indication catch-up
            last_mac += 1 (wrapped)
            DeliverToMAC(last_mac)
        end while
        ID_DEC += 1 (wrapped)
    end while
end procedure
```

C. Convergence Optimization Policy

To counteract fluctuating network latency and maintain scheduling robustness, this paper proposes a dynamic convergence optimization policy. Upon receiving the timing statistics reported by the PNF, the system incorporates an Exponentially Weighted Moving Average (EWMA) model to estimate the mean latency $\bar{L}$ and jitter variance V_{jitter}. Given the worst-case late arrival L_{worst} in the current observation period, the iterative update formulas are:

$$\bar{L}^{(k)} = \bar{L}^{(k-1)} + \frac{L_{\text{worst}} - \bar{L}^{(k-1)}}{8} \tag{3}$$

$$V_{\text{jitter}}^{(k)} = V_{\text{jitter}}^{(k-1)} + \frac{|L_{\text{worst}} - \bar{L}^{(k-1)}| - V_{\text{jitter}}^{(k-1)}}{4} \tag{4}$$

Before making tracking adjustments, a dynamic safety margin M_{safe} is established to cover the networking jitter, and an absolute boundary B_{abs} is enforced to limit overly aggressive delays:

$$M_{\text{safe}} = \max\left(1.5 \cdot T_{\text{slot}}, 1.5 \cdot V_{\text{jitter}}\right) \tag{5}$$

$$B_{\text{abs}} = 2 \cdot T_{\text{slot}} \tag{6}$$

Furthermore, the algorithm designs multiple Panic Criteria triggers (including deadline edge $panic_{\text{edge}}$, severe jitter $panic_{\text{jitter}}$, statistical anomaly $anomaly_{\text{stat}}$, and strict violation $violation_{\text{strict}}$). To prevent unexpected oscillation during target transitions, a Directionality Guard is introduced. Additionally, a Penalty Counter & Anti-bounce mechanism filters out pseudo-stable low-latency states to prevent prematurely lowering s_{ahead}. When panic events are detected, the algorithm initiates a fast-attack mode, radically increasing S_{target} to ensure connection continuity. Conversely, when redundant phase advance exists continuously without penalties, a slow-decay backoff is triggered. The overall dynamic adjustment is condensed in Algorithm 3.

Algorithm 2 Delay Manager: Dynamic Scheduler Trigger Timing

Require:
t_{late}: Worst late timing from PHY (Input)
T_{slot}: Slot duration in microseconds
S_{curr}: Current slots ahead (Environment state)
Ensure:
S_{next}: Adjusted slots ahead (Output: Higher = Earlier, Lower = Later)

Update EWMA Mean Late (E) and Jitter Variance (V) using t_{late}
$B_{safe} \leftarrow (S_{curr} - 1) \times T_{slot}$ ▷ Absolute safe boundary
$C_{panic} \leftarrow (t_{late} \geq T_{slot}) \vee (V >$ SafeMargin$) \vee$ (Statistical Anomaly)
if C_{panic} then
    $S_{next} \leftarrow S_{curr} +$ StepUp ▷ Earlier: Fast attack due to violation/jitter
    Reset safety counters and apply reduction penalty
else if $t_{late} < -B_{safe}$ and Reduction is not locked then
    $Count_{safe} \leftarrow Count_{safe} + 1$
    if $Count_{safe} >$ Decay Threshold then
        $S_{next} \leftarrow S_{curr} - 1$ ▷ Later: Proactive safe latency reduction
        $Count_{safe} \leftarrow 0$
    end if
else
    $S_{next} \leftarrow S_{curr}$ ▷ Maintain: Timing is stable within bounds
end if
$S_{next} \leftarrow \max(1, \min(S_{next}, 8))$ ▷ Apply absolute system bounds
return S_{next}

Algorithm 3 Dynamic Convergence Optimization Policy

procedure ConvergenceOpt($L_{\text{worst}}, T_{\text{slot}}$)
    $diff \leftarrow L_{\text{worst}} - \bar{L}$
    $\bar{L} \leftarrow \bar{L} + diff/8$
    $V_{\text{jitter}} \leftarrow V_{\text{jitter}} + (|diff| - V_{\text{jitter}})/4$
    $B_{\text{abs}} \leftarrow 2 \cdot T_{\text{slot}}$, $S_{\text{target}} \leftarrow s_{\text{ahead}}$
    Evaluate logic flags: $panic$, $anomaly_{\text{stat}}$
    if $L_{\text{worst}} < 0$ then
        $anomaly_{\text{stat}} \leftarrow false$
    end if
    $\Delta T_{\text{adv}} \leftarrow T_{\text{adv}} - last\_T_{\text{adv}}$
    $dir\_helpful \leftarrow \text{SignMatch}(\Delta T_{\text{adv}}, L_{\text{worst}})$
    $S_{\text{prop}} \leftarrow \text{clamp}(\text{round}(T_{\text{adv}}/T_{\text{slot}}), 1, S_{\max})$
    if $L_{\text{worst}} \geq T_{\text{slot}}/2$ then
        $S_{\text{prop}} \mathrel{+}= 1$
    end if
    if $S_{\text{prop}} \neq S_{\text{target}}$ then
        if $|S_{\text{prop}} - S_{\text{target}}| > 1$ and not $dir\_helpful$ then
            $S_{\text{target}} \mathrel{+}= \text{sign}(S_{\text{prop}} - S_{\text{target}})$
        else
            $S_{\text{target}} \leftarrow S_{\text{prop}}$
        end if
    end if
    if $panic$ or $anomaly_{\text{stat}}$ then ▷ Fast-Attack
        $step \leftarrow (violation_{\text{strict}})?\lfloor L_{\text{worst}}/T_{\text{slot}} \rfloor + 2 : 2$
        if $violation_{\text{strict}}$ then
            $penalty \mathrel{+}= 10000$
        end if
        $S_{\text{target}} \leftarrow \min(S_{\text{target}} + step, S_{\max})$
        $safe\_cnt \leftarrow 0$
    else if $L_{\text{worst}} < -B_{\text{abs}}$ then ▷ Slow-Decay
        $safe\_cnt \mathrel{+}= 1$
        $penalty \leftarrow \max(0, penalty - 1)$
        if $safe\_cnt > 2000 + penalty$ then
            Anti-bounce top-tier hold logic once
            $S_{\text{target}} \mathrel{-}= 1$
            $safe\_cnt \leftarrow 0$
        end if
    else
        Reset counters and top-hold state
    end if
    $S_{\text{target}} \leftarrow \text{clamp}(S_{\text{target}}, 1, S_{\max})$
    if $S_{\text{target}} \neq s_{\text{ahead}}$ then ▷ Phase drift compensation
        $\bar{L} \mathrel{+}= (S_{\text{target}} - s_{\text{ahead}}) \cdot T_{\text{slot}}$
        $last\_T_{\text{adv}} \leftarrow T_{\text{adv}}$, $s_{\text{ahead}} \leftarrow S_{\text{target}}$
    end if
end procedure

IV. Experimental Results

To validate the hypothesis and the effectiveness of the proposed dynamic trigger scheduler algorithm, we systematically designed multiple experimental scenarios. Section IV-A outlines the general software and hardware

TABLE I
Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

environment used across all tests. Subsequent sections present specific scenarios: Scenario I (IV-B) isolates and verifies the problems caused by non-deterministic latency under standard OpenAirInterface implementations, while Scenario II (IV-C) and subsequent experiments evaluate the robustness of our proposed time-advance control under varying levels of congestion.

A. Experimental Scenario

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

B. Scenario I: Analysis of Non-Deterministic Latency

Deploying the VNF and PNF on different machines introduces significant challenges. Because the MAC scheduler is separated from the PHY layer, strict slot synchronization is required between these distributed components. However, even when deployed in the exact same network environment, different traffic loads directly impact the latency. Figure 2 illustrates the latency between the VNF scheduler and the PNF under varying traffic loads. As traffic increases, three factors heighten the overall delay: the scheduler requires more processing time, the data packing time lengthens, and the VNF-to-PNF One-Way Delay (OWD) is affected. Consequently, the VNF cannot reliably transmit data exactly at the slot boundary. Instead, it must dynamically advance its scheduling point to compensate for these varying delays and ensure packets arrive at the PNF on time.

C. Scenario II: [Placeholder for Future Experiment]

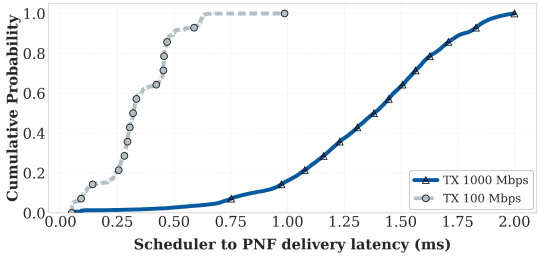

Fig. 2. Observation of latency between VNF scheduler and PNF under different traffic loads.

References

[1] 3GPP, "Ng-ran; architecture description," 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 38.401, 2024, version 18.0.0.

[2] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits in 5g next-generation radio access networks," IEEE Communications Surveys & Tutorials, vol. 21, no. 3, pp. 2349–2372, 2019.

[3] Small Cell Forum, "5g nfapi specification," Small Cell Forum (SCF), Tech. Rep. SCF222, 2023. [Online]. Available: https://www.smallcellforum.org/

[4] O-RAN Alliance, "O-ran fronthaul working group control, user and synchronization plane specification," O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v11.00, 2023.

[5] IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems, IEEE Std. IEEE 1588-2019, 2019.

[6] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, "A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond," IEEE Communications Surveys & Tutorials, vol. 25, no. 4, pp. 2791–2832, 2023.

[7] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. D. Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, "A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges," IEEE Communications Surveys & Tutorials, vol. 28, pp. 1637–1678, 2026.

[8] S. C. Forum, "5g network fapi (nfapi) specification," Small Cell Forum, Tech. Rep. SCF225, 2020.

[9] F. Mehran and R. MacKenzie, "Experimental evaluation of multi-vendor virtualized ran using non-ideal fronthaul," in 2020 IEEE 31st Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC). IEEE, 2020.

[10] B. N. Khalaf, W. H. Ali, R. S. Alhumaima, and H. A. J. Alshamary, "Delay aware resource allocation in oran through network optimization," IET Wireless Sensor Systems, 2024.

[11] M. Huang and X. Zhang, "Distributed mac scheduling scheme for c-ran with non-ideal fronthaul in 5g networks," in Communication Infrastructure Research Lab (Intel Labs China). IEEE, 2026.

[12] B. Akgun et al., "Interference-aware intelligent scheduling for virtualized private 5g networks," IEEE Access, vol. 12, 2024.

[13] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," IEEE Communications Standards Magazine, vol. 2, no. 3, pp. 32–40, 2018.

[14] M. S. Lohith et al., "Design and implementation of nfapi for 5g nr small cells on fpga-based soc," in 2021 IEEE 18th India Council International Conference (INDICON), 2021, pp. 1–6.

[15] L. J. Brown, L. H. Crockett, and R. W. Stewart, "A single-chip split-6 phy implementation for 5g nr, using rfsoc and pynq," Franklin Open, vol. 13, p. 100408, 2025.

[16] X. Wang et al., "An open-source implementation of the 5g nr functional split option 6," in 2022 IEEE International Conference on Communications (ICC), 2022, pp. 1–6.